SparseDR: Differentiable Rendering of Sparse Signed Distance Fields

First Author[1,2] , **Second Author**[2] , **Third Author**[1,2] and
[1]First Affiliation
[2]Second Affiliation
third@other.example.com, fourth@example.com

Abstract

We present SparseDR, a differentiable rendering implementation designed for sparse representations based on Signed Distance Fields (SDF). We leverage the Sparse Brick Set (SBS) representation and propose an adaptation of redistancing and regularization of the SDF defined on SBS. This enables our method to surpass existing differentiable rendering implementations in accuracy at limited memory budgets by increasing the effective resolution of the SDF representation.

1 Introduction

Signed distance function (SDF) provide a continuous, implicit representation of geometry, defining the shortest Euclidean distance from any point in space to the nearest surface, with the sign indicating interior or exterior regions. This enables gradient-based optimization by supporting differentiable rendering, where ray marching samples distances to evaluate color, facilitating inverse rendering tasks such as shape and material recovery from images [Vicini *et al.*, 2022; Wang *et al.*, 2024; Zhang *et al.*, 2025].

In an SDF-based differentiable rendering, each gradient descent step updates the distance values at grid cells, so the grid no longer strictly satisfies the SDF property. To address this issue, existing methods typically apply *redistancing* — a process that recomputes grid values so that each cell stores the correct distance to the surface [Detrixhe *et al.*, 2013]. This makes SDF-based differentiable rendering on sparse data structures non-trivial, so most existing methods rely on regular grids. Unfortunately, this approach results in high memory and computational costs, while the reconstruction accuracy remains limited by the maximum grid resolution. Our work proposes a solution to this problem, alleviating limitations through better data structures and sampling techniques:

- We propose an adaptation of SDF-based differentiable rendering to a sparse data structure, which increases the effective resolution of the SDF representation and thereby improves the surface reconstruction accuracy.
- We adapt redistancing to sparse data structures by separating it into two distinct algorithms: local and global redistancing.

2 Related Work

The representation of SDF on a regular grid in existing works [Vicini *et al.*, 2022; Wang *et al.*, 2024; Zhang *et al.*, 2025] has three major limitations: (1) significant memory consumption, especially when using automatic differentiation frameworks such as DrJit [Jakob *et al.*, 2022] or PyTorch [Jason and Yang, 2024]; (2) slow ray–surface intersection based on sphere tracing; (3) an expensive redistancing algorithm that propagates modified SDF values across the entire grid. Our work shows how these components – data structure, ray intersection, and redistancing – can be joined together in an efficient manner.

3 Implementation Details

SparseDR uses the Sparse Brick Set (SBS) proposed in [Söderlund *et al.*, 2022], a hybrid of hierarchical and regular representations (Fig. 1). The entire scene is encoded as a BVH, while its leaves store small regular grids, for example 4x4x4 voxels. For gradient computation, we adapt the relaxed boundary method from [Wang *et al.*, 2024], which we refer to as our baseline.

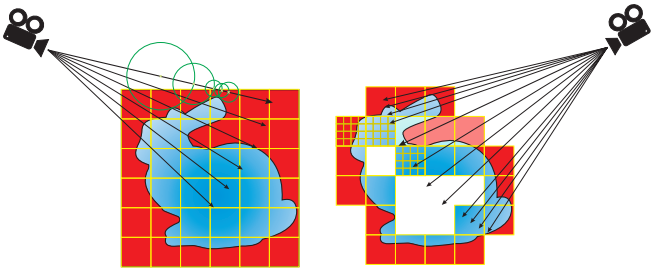

Figure 1: The baseline (on the left) uses dense SDF grid to represent the surface, sphere tracing for ray-scene intersection and uniform image sampling when shooting rays. SparseDR (on the right) uses Sparse Brick Set where a ray intersects each brick on its way until it hits the surface. It also uses adaptive sampling for boundary integral placing more rays in the relaxed boundary area. Red color depicts positive-defined SDF values, blue-colored — negative.

At the beginning of the optimization process, the distance function is initialized with a sphere defined on a low-resolution grid (32x32x32). At each iteration of an epoch, we estimate the gradient and update the distance values using a gradient descent step, followed by a regularization and local

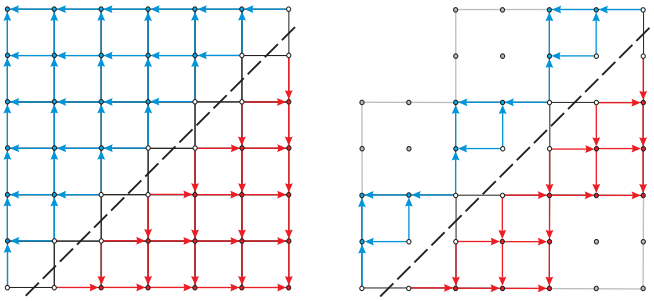

Figure 2: Comparison of the global redistancing algorithm on dense grid (on the left) with the local redistancing (on the right) on SBS, bricks are 3x3 distances in size. The dots represent the distance values. Red and blue dots are the distances outside and inside the object respectively. Dashed line represents the surface, set by distances colored white. Arrows show the order of updating the values. Bricks without surface (grey) are ignored by accelerated redistancing.

redistancing (see Figure 2). On the end of each epoch two steps are executed: upsampling step and global redistancing with the exception that the final iteration does not include upsampling.

Gradient evaluation. The key distinction of the method underlying SparseDR lies in the transition from an SDF grid to a sparse brick set. In our case, each brick has a fixed size of 4×4×4 voxels storing 5x5x5 distances. For the SBS, a BVH tree is constructed, with each leaf node storing a single brick. Instead of sphere tracing we have adapted Newton method from [Söderlund *et al.*, 2022] for ray–SDF intersection. A ray first traverses the BVH; if an intersection with a leaf node is found, it then intersects the voxels of the brick. The ray traverses each brick along its path until it hits the surface or there are no bricks left. The Newton method was modified to search for relaxed boundary points: within the voxel, a potential relaxed boundary point is found as the root of a quadratic equation, after which the SDF value at the point is checked to be less than the relaxed epsilon. The case when the local minimum occurs on the boundary between voxels or bricks is handled separately.

SparseDR is implemented in C++ and Vulkan and does not use automatic differentiation, i.e. all derivatives were obtained manually. In the shader we do 16 samples per pixel and store information which voxels was hit by the ray in local array. Then we average the color for 16 samples, compute image gradients and backpropagate them for each of 16 samples to bircks using atomic operations. SparseDR optionally uses an additional pass that samples only pixels containing boundaries and evaluates only the boundary integral, without the need for color computation. This adaptive boundary sampling allows cheaper samples and yields more accurate gradients for smaller relaxed epsilon values.

Redistancing. We implement two versions of redistancing: local and global (Fig. 2). The local version is applied at each optimization step and recomputes distances only within individual bricks. In practice, the local redistancing is almost always sufficient, since our ray–surface intersection method relies only on the distances within the traversed bricks. In other words, for the proposed method, the function is not required to strictly satisfy the SDF property during optimization. However, this approach may cause distance inconsistencies between adjacent bricks, so we perform global redistancing like in at the beginning of each epoch. The global redistancing addresses distance updates across empty space between bricks via extrapolation between brick faces. For each face lacking a direct neighbor, the nearest neighbor bricks are identified, and one of their faces is selected for extrapolation. These neighbors are determined once; afterward, values are extrapolated after each fast sweeping step, accounting for grid spacing. The minimum absolute value from extrapolated and prior values is selected. The fast sweeping method [Detrixhe *et al.*, 2013] is applied as the final step to propagate updated distances.

Sparsification and Upsampling. The sparsification step is performed before upsampling. During this stage, all bricks containing the surface are preserved, as well as all bricks within a radius of 1 voxel from them, including diagonal neighbors. If any of the adjacent bricks are missing, they are added at this stage. This is necessary in cases where the reconstructed object contains fine details, so that the bricks where these details' reconstruction should reside would otherwise be absent. Thus, this step not only contributes to reducing the model size, but also eliminates a potential limitation compared to the baseline method. After the sparsification step, the global redistancing is applied to recompute all distances before upsampling or saving the resulting SBS, ensuring that the model represents a valid SDF. The upsampling step doubles the resolution of the remaining bricks, generating eight new bricks from each original one. The new distance values are computed using trilinear interpolation.

4 Experiments

Experiments with the baseline method were conducted using the official implementation from [Wang, 2024]. The baseline method employed the hyperparameters specified in the paper [Wang *et al.*, 2024], while the remaining parameters were taken from the configuration file "turbo.json". The only modifications concerned the number of viewpoints, the batch size, and the upsampling iterations. For consistency across experiments, an identical lighting setup was used: two directional light sources with directions $(1, 1, 1)$ and $(-1, -1, -1)$, along with ambient light. The experiments were carried out on six models. Reconstruction used 16 viewpoints uniformly distributed over a sphere around the scene, generated by the algorithm from [Wang, 2024]. The batch size is 4 viewpoints, image resolution is 1024×1024. The full optimization process comprised 1000 steps, with upsampling applied at steps 200, 400, 600, 700, 800, and 900. Computations were performed on an AMD Ryzen 9 7950X CPU and an NVIDIA GeForce RTX 4090 GPU.

Figure 3 shows the average reconstruction time across all models for each stage between upsampling steps for SparseDR and the baseline method. The stages are labeled by the SDF grid resolution in voxels corresponding to each level of detail. "Rendering" refers to consecutive rendering of all views within the current batch. Figure 4 presents the mean reconstruction quality across viewpoints for SparseDR and the

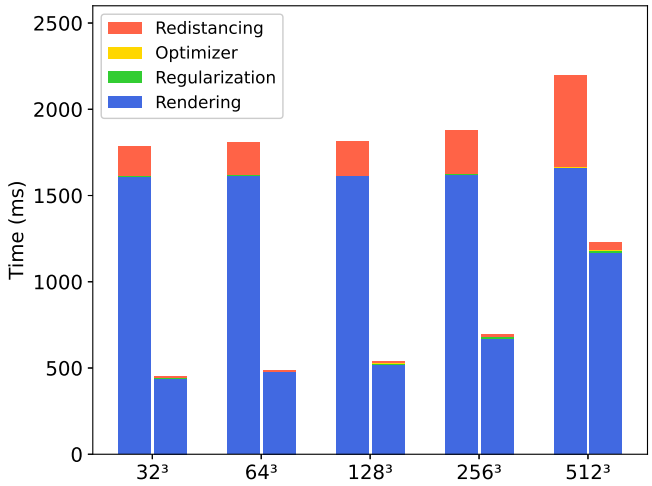

Figure 3: Average time of 1 iteration. For each grid size, the left bar shows the baseline, the right – SparseDR for the SBS of equivalent level of detail.

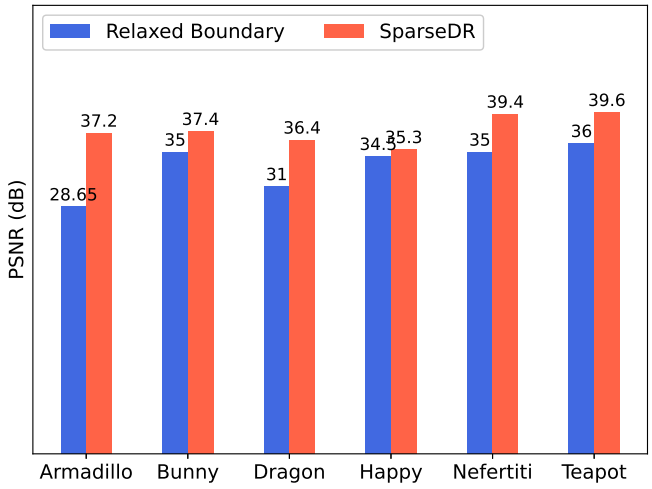

Figure 4: Average PSNR values across viewpoints for models.

Model	32^3	128^3	512^3	1024^3	2048^3
Baseline	0.125	8	512	–	–
Ours(Armadillo)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Bunny)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Dragon)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Happy)	0.5	6.3	96.4	392.1	1597.7
Baseline	0.125	8	512	–	–
Ours(Nefertiti)	0.5	6.3	96.4	392.1	1597.7
Baseline	**0.125**	8	512	–	–
Ours(Teapot)	0.5	**6.3**	**96.4**	**392.1**	**1597.7**

Table 1: Model sizes for Relaxed Boundary and SparseDR methods. All values are in MB.

baseline method. Finally, Figure 1 illustrates the model sizes for SparseDR and the baseline method obtained after each upsampling step. For the baseline method it shows the grid size, and for SparseDR both the size of the SBS and of its BVH. The results demonstrate that the growth of the SBS size is an order of magnitude smaller than that of the SDF grid.

5 Conclusion

In this work we present SparseDR, a system for 3D reconstruction based on the method of differentiable rendering for a sparse SDF representation. Using [Wang *et al.*, 2024] as a baseline, we leverage the advantages of SBS and modify all stages of the algorithm. As a result, SparseDR achieves performance and reconstruction quality comparable to the original approach while reducing memory consumption by an order of magnitude. Future work will focus on upgrading SparseDR to a full-fledged path tracer and addressing the challenges that complicate the practical use of differentiable rendering, in particular the need for precise camera parameter information.

References

[Detrixhe *et al.*, 2013] Miles Detrixhe, Frédéric Gibou, and Chohong Min. A parallel fast sweeping method for the eikonal equation. *Journal of Computational Physics*, 237:46–55, 2013.

[Jakob *et al.*, 2022] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. Dr.jit: a just-in-time compiler for differentiable rendering. 41(4), 2022.

[Jason and Yang, 2024] Jason and Edward He et al. Yang. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 929–947, New York, NY, USA, 2024. Association for Computing Machinery.

[Müller *et al.*, 2022] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 41(4), 2022.

[Söderlund *et al.*, 2022] Herman Hansson Söderlund, Alex Evans, and Tomas Akenine-Möller. Ray tracing of signed distance function grids. *Journal of Computer Graphics Techniques Vol*, 11(3), 2022.

[Vicini *et al.*, 2022] Delio Vicini, Sébastien Speierer, and Wenzel Jakob. Differentiable signed distance function rendering. *ACM Trans. Graph.*, 41(4), July 2022.

[Wang *et al.*, 2024] Zichen Wang, Xi Deng, Ziyi Zhang, Wenzel Jakob, and Steve Marschner. A simple approach to differentiable rendering of sdfs. In *SIGGRAPH Asia 2024 Conference Papers*, New York, NY, USA, 2024. Association for Computing Machinery.

[Wang, 2024] Zichen Wang. A simple approach to differentiable rendering of sdfs repository, 2024.

[Zhang *et al.*, 2025] Ziyi Zhang, Nicolas Roussel, and Wenzel Jakob. Many-worlds inverse rendering. *ACM Trans. Graph.*, 45(1), October 2025.